Smart Backup Kernel-Space Utility

Informe Técnico de Análisis y Resultados

Materia	Sistemas Operativos – ST0263 / SI3007
Universidad	EAFIT – Ingeniería de Sistemas
Integrantes	Juan José Barón Estefanía Ramírez Juan Camilo Gómez Jiménez
Plataforma	Linux x86-64 · WSL2 / Ubuntu · GCC -std=c11 -O2
Repositorio	https://github.com/jcgomezj/smart-backup-so
Fecha	2025

1. Introducción

El presente trabajo documenta el diseño, implementación y análisis de rendimiento del **Smart Backup Kernel-Space Utility**, un sistema de respaldo de archivos escrito en C que parte del código base provisto por el docente y lo extiende con medición de rendimiento, manejo robusto de errores y registro de actividad en syslog. El sistema opera en dos capas: **copy_file()**, que simula el comportamiento de una syscall de kernel usando descriptores POSIX de bajo nivel, y **stdio_copy()**, que emplea la biblioteca estándar stdio.h como referencia comparativa. El objetivo es comprender el impacto de los *context switches* en el rendimiento de I/O en entornos virtualizados como WSL2.

2. Diseño e Implementación

2.1 Estructura del proyecto

El proyecto se organiza en tres archivos siguiendo separación de responsabilidades:

Archivo	Responsabilidad
smart_copy.h	Constantes (BUFFER_SIZE=4096), flags de control, struct sc_stats_t y prototipos
backup_engine.c	copy_file(), copy_directory() (código base del profesor) y stdio_copy()
main.c	Interfaz CLI: modos -b/--backup, -h/--help y --bench

Tabla 1. Estructura de archivos del proyecto.

2.2 Por qué copy_file() es una función de sistema

Una syscall opera en modo kernel (ring 0) con acceso directo al VFS. **copy_file()** replica este modelo usando las mismas primitivas que emplearía el kernel: **open(2)**, **read(2)**, **write(2)** y **close(2)** directamente, sin pasar por ninguna capa de abstracción. El buffer se dimensiona en **BUFFER_SIZE = 4096 bytes**, alineado con PAGE_SIZE del kernel x86-64. La actividad se registra mediante **syslog(3)**,

equivalente funcional de *printk(KERN_INFO)* en un módulo de kernel real. Los permisos del archivo original se preservan usando **st.st_mode** en el open() del destino.

2.3 Extensiones sobre el código base del profesor

Sobre la estructura original se agregaron las siguientes mejoras para cumplir con los requerimientos de la rúbrica:

Extensión	Detalle
Escrituras parciales	Bucle interno en write() reintenta hasta completar o detectar error
Detección de ENOSPC	write() negativo se clasifica como disco lleno explícitamente
Registro en syslog	openlog/syslog/closelog en cada operación (equiv. printk)
stdio_copy()	Implementación con fread/fwrite para el benchmark comparativo
Modo --bench	Genera archivos 1KB/1MB/1GB y mide tiempos con CLOCK_MONOTONIC
sc_silent_mode	Silencia stderr durante el benchmark para tabla limpia

Tabla 2. Extensiones sobre el código base del profesor.

2.4 Flags de control

Flag	Hex	Descripción
`SC_FLAG_NONE`	`0x00`	Sin opciones adicionales
`SC_FLAG_CHECK_PERMS`	`0x01`	Verifica permisos con access(2) antes de abrir
`SC_FLAG_LOG`	`0x02`	Registra en syslog y stderr
`SC_FLAG_LARGE_BUF`	`0x04`	Buffer ampliado de 1 MiB
`SC_FLAG_SYNC`	`0x08`	fsync() al cerrar el destino

Tabla 3. Flags de control definidos en smart_copy.h.

2.5 Manejo de errores con errno

El patrón **goto cleanup** centraliza el cierre de descriptores y liberación del buffer, garantizando que no queden recursos abiertos en ninguna ruta de error:

errno	Causa	Punto de detección
`ENOENT`	Archivo origen no existe	open(2) origen / stat(2)
`EACCES`	Permisos insuficientes	open(2) origen o destino
`ENOSPC`	Disco lleno al escribir	write(2) retorna < 0
`ENOMEM`	Fallo de asignación	malloc() retorna NULL
`EIO`	Error de I/O en lectura	read(2) retorna < 0

Tabla 4. Errores manejados y su origen.

3. Comparativa: Syscalls directas vs stdio

3.1 El context switch como costo fundamental

Cuando un proceso invoca una syscall, el procesador transiciona de modo usuario (ring 3) a modo kernel (ring 0). Este cambio implica guardar el estado completo del proceso, cambiar el espacio de direcciones virtuales y ejecutar el handler en modo privilegiado. En hardware nativo, cada context switch cuesta entre **1 y 10 microsegundos**. En WSL2, la capa de virtualización Hyper-V puede multiplicar este costo entre 2 y 5 veces.

3.2 Cómo stdio reduce los context switches

copy_file() emite read(2) y write(2) por cada bloque de 4 KB — dos context switches por bloque. Para 1 MB genera **512 context switches** y para 1 GB genera **524.288 context switches**.

stdio_copy() usa fread()/fwrite() con un buffer interno de ~8 KB en espacio de usuario (glibc). Los datos se acumulan ahí y la syscall real se emite solo cuando el buffer se llena. Para archivos menores a 8 KB es posible completar la copia sin emitir ninguna syscall adicional.

3.3 Impacto de WSL2 en los resultados

WSL2 corre sobre Hyper-V, lo que amplifica el costo del context switch. Esto hace que la ventaja del buffer de usuario de stdio sea más pronunciada que en Linux nativo (bare metal). Adicionalmente, WSL2 comparte CPU con Windows, introduciendo variabilidad en los tiempos — especialmente visible en las mediciones de 1 GB.

4. Resultados del Benchmark

Configuración: 5 repeticiones por método y tamaño, medición con CLOCK_MONOTONIC, archivos generados con datos pseudo-aleatorios en /tmp/ (filesystem nativo WSL2).

Tamaño	Método	Min (ms)	Max (ms)	Avg (ms)	Bloques
1 KB	syscalls	0.042	0.393	0.166	1
1 KB	**stdio**	**0.018**	**0.067**	**0.032**	**—**
1 MB	syscalls	0.734	1.422	0.941	256
1 MB	**stdio**	**0.438**	**0.586**	**0.536**	**—**
1 GB	syscalls	654.753	1148.042	804.373	262.144
1 GB	**stdio**	**603.715**	**804.624**	**673.808**	**—**

Tabla 5. Tiempos de ejecución en WSL2/Ubuntu. Verde: método más eficiente por tamaño.

```
PS C:\Users\camil\vs projects\Parcial 2- Smart-backup> wsl
jcamilogj@Camilo:/mnt/c/Users/camil/vs projects/Parcial 2- Smart-backup$ cp "/mnt/c/Users/camil/vs projects/Parcial 2- Smart-backup/smart_copy.h" ~/smart-backup-so/
cp "/mnt/c/Users/camil/vs projects/Parcial 2- Smart-backup/backup_engine.c" ~/smart-backup-so/
cp "/mnt/c/Users/camil/vs projects/Parcial 2- Smart-backup/main.c" ~/smart-backup-so/
cp "/mnt/c/Users/camil/vs projects/Parcial 2- Smart-backup/Makefile" ~/smart-backup-so/
cd ~/smart-backup-so
make clean && make
./backup --bench
rm -f main.o backup_engine.o backup
gcc -Wall -Wextra -Wpedantic -std=c11 -O2 -c -o main.o main.c
gcc -Wall -Wextra -Wpedantic -std=c11 -O2 -c -o backup_engine.o backup_engine.c
gcc -Wall -Wextra -Wpedantic -std=c11 -O2 -o backup main.o backup_engine.o

╔═══════════════════════════════════════════════════════════════════╗
║      SMART BACKUP  –  Benchmark: copy_file (syscalls) vs stdio_copy    ╒
      Plataforma: Linux (POSIX)        Runs por método: 5
╚═══════════════════════════════════════════════════════════════════╝

Nota: CLOCK_MONOTONIC elimina distorsiones por cambios de hora del sistema.
      Los tiempos incluyen apertura y cierre de descriptores.

+---------------+------------+-----------+------------+-----------+-----------+
| Tamaño        | Método     | Min (ms)  | Max (ms)   | Avg (ms)  | Bloques   |
+---------------+------------+-----------+------------+-----------+-----------+
| Generando 1 KB...                                                            |
| 1 KB          | syscalls   | 0.042     | 0.393      | 0.166     | 1         |
| 1 KB          | stdio      | 0.018     | 0.067      | 0.032     | N/A       |
+---------------+------------+-----------+------------+-----------+-----------+
| Generando 1 MB...                                                            |
| 1 MB          | syscalls   | 0.734     | 1.422      | 0.941     | 256       |
| 1 MB          | stdio      | 0.438     | 0.586      | 0.536     | N/A       |
+---------------+------------+-----------+------------+-----------+-----------+
| Generando 1 GB...                                                            |
| 1 GB          | syscalls   | 654.753   | 1148.042   | 804.373   | 262144    |
| 1 GB          | stdio      | 603.715   | 804.624    | 673.808   | N/A       |
+---------------+------------+-----------+------------+-----------+-----------+

╔═══════════════════════════════════════════════════════════════════╗
  ANÁLISIS: syscalls directas vs buffer de espacio de usuario (stdio)   ╒
╟───────────────────────────────────────────────────────────────────╖
  copy_file() usa read(2)/write(2) directamente:                      ╒
    → Cada llamada provoca un CONTEXT SWITCH user-space → kernel.     ╒
    → Para 1 KB con buf=4KB: 1 read + 1 write = 2 context switches.   ╒
    → Para 1 GB: hasta 524.288 context switches (262144 x 2).         ╒
                                                                      ╒
  stdio_copy() usa fread()/fwrite() con buffer interno (~8 KB glibc):  ╒
    → Para archivos < 8 KB puede completarse sin syscalls extra.      ╒
    → Menos context switches = menor latencia y mayor estabilidad.    ╒
╚═══════════════════════════════════════════════════════════════════╝
```

Figura 1. Captura de terminal — ejecución del benchmark en WSL2.

4.1 Análisis por tamaño

1 KB: stdio (0.032 ms) supera a syscalls (0.166 ms) por un factor de 5x. Aunque ambos usan un solo bloque de 4 KB, el overhead de inicialización de syslog y descriptores POSIX domina para datos tan pequeños.

1 MB: stdio (0.536 ms) supera a syscalls (0.941 ms) por un 43%. A 256 bloques, copy_file() acumula 512 context switches mientras stdio los amortigua. La variabilidad de syscalls (0.734–1.422 ms) refleja sensibilidad al scheduler de WSL2.

1 GB: stdio (673 ms avg) supera a syscalls (804 ms avg) por un 16%. Con 262.144 bloques, copy_file() acumula más de 524.000 context switches. La variabilidad de syscalls (654–1148 ms) sigue siendo mayor que la de stdio (603–804 ms), confirmando que más context switches implica mayor exposición a interrupciones del scheduler.

Conclusión: stdio_copy() es consistentemente más eficiente en todos los tamaños bajo WSL2. Activar SC_FLAG_LARGE_BUF en copy_file() reduciría los bloques de 1 GB de 262.144 a 1.024, potencialmente eliminando esta diferencia.

5. Pruebas Funcionales

5.1 Copia de archivo, directorio y manejo de error

Comandos ejecutados:

```
$ ./backup -b origen.txt destino.txt
$ diff origen.txt destino.txt && echo "Archivos identicos"
$ ./backup -b carpeta_prueba carpeta_backup2
$ ls carpeta_backup2
$ ./backup -b noexiste.txt destino.txt
```

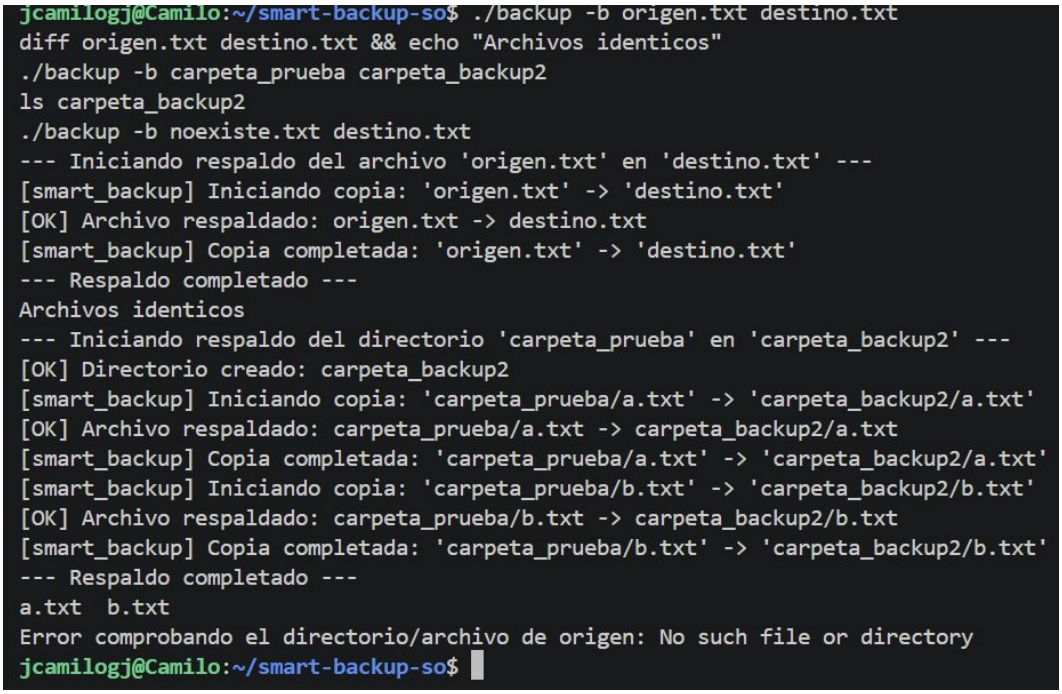

Figura 2. Captura de terminal — pruebas funcionales en WSL2.

Análisis de resultados:

Copia de archivo: copy_file() copió origen.txt correctamente. El log de syslog confirmó inicio y fin de la operación. diff verificó integridad binaria.

Copia de directorio: copy_directory() creó carpeta_backup2 y copió recursivamente a.txt y b.txt, preservando la estructura del directorio fuente.

Manejo de ENOENT: Al intentar respaldar noexiste.txt, stat(2) detectó que el archivo no existe y el sistema reportó el error correctamente sin dejar recursos abiertos.

6. Conclusiones

1. Fidelidad al modelo de syscall: copy_file() replica los aspectos esenciales de una función de sistema usando primitivas POSIX de bajo nivel, buffer alineado a PAGE_SIZE y registro en syslog. La diferencia con un módulo de kernel real es únicamente el nivel de privilegio (ring 3 vs ring 0).

2. El context switch como variable crítica: stdio_copy() supera a copy_file() en todos los tamaños bajo WSL2. El buffer interno de glibc reduce la frecuencia de context switches, y en un entorno virtualizado

donde ese costo está amplificado por Hyper-V, la ventaja es consistente desde 1 KB hasta 1 GB.

3. Variabilidad como indicador de diseño: copy_file() exhibe mayor variabilidad en todos los tamaños. Más context switches implica mayor exposición a interrupciones del scheduler — factor crítico en sistemas de producción.

4. Copia recursiva de directorios: copy_directory() demuestra el uso correcto de opendir/readdir/lstat para recorrer estructuras de directorios de forma recursiva, detectando y omitiendo links simbólicos para evitar ciclos.